7 – Verificación de consistencia con Streamlit

El enunciado exige demostrar que la app de Streamlit produce **las mismas predicciones** que la pipeline cargada directamente. A continuación se evalúan dos instancias nuevas con el modelo cargado desde disco, para que puedan compararse con las predicciones de la app.

+ Código + Texto

```python
# Instancia 1 – cliente no contactado previamente (pdays original = -1)
# El PdaysTransformer del pipeline creará contacted_before=0 y pondrá pdays=0
instancia_1 = pd.DataFrame([{
    'age': 36, 'job': 'technician', 'marital': 'married',
    'education': 'secondary', 'default': 'no', 'balance': 1466,
    'housing': 'yes', 'loan': 'no', 'contact': 'unknown',
    'day': 29, 'month': 'may', 'duration': 441, 'campaign': 1,
    'pdays': -1, 'previous': 0, 'poutcome': 'unknown'
}])

# Instancia 2 – cliente contactado previamente (pdays original = 389)
# El PdaysTransformer del pipeline creará contacted_before=1
instancia_2 = pd.DataFrame([{
    'age': 41, 'job': 'entrepreneur', 'marital': 'married',
    'education': 'primary', 'default': 'no', 'balance': 0,
    'housing': 'no', 'loan': 'no', 'contact': 'cellular',
    'day': 3, 'month': 'jul', 'duration': 438, 'campaign': 2,
    'pdays': 389, 'previous': 1, 'poutcome': 'other'
}])

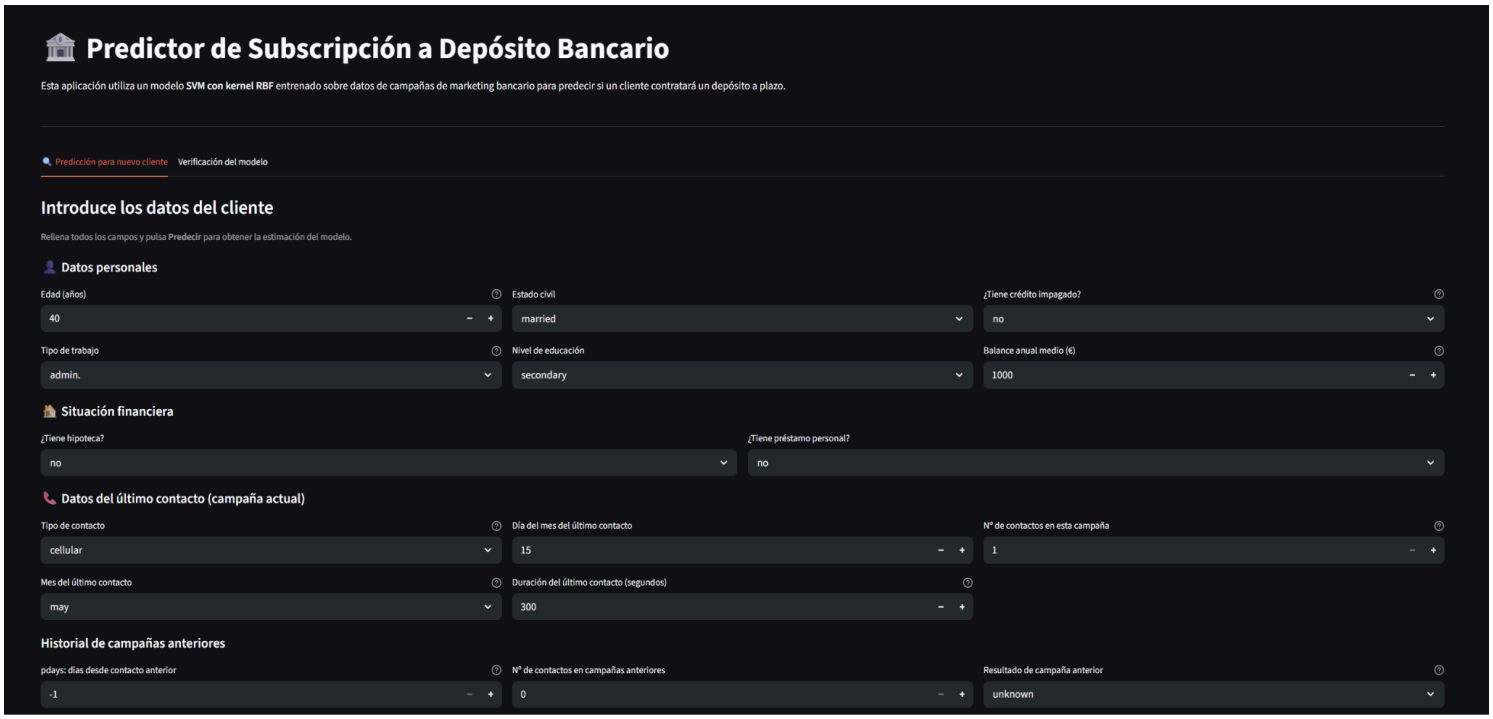
pred_1 = modelo_recargado.predict(instancia_1)[0]
pred_2 = modelo_recargado.predict(instancia_2)[0]

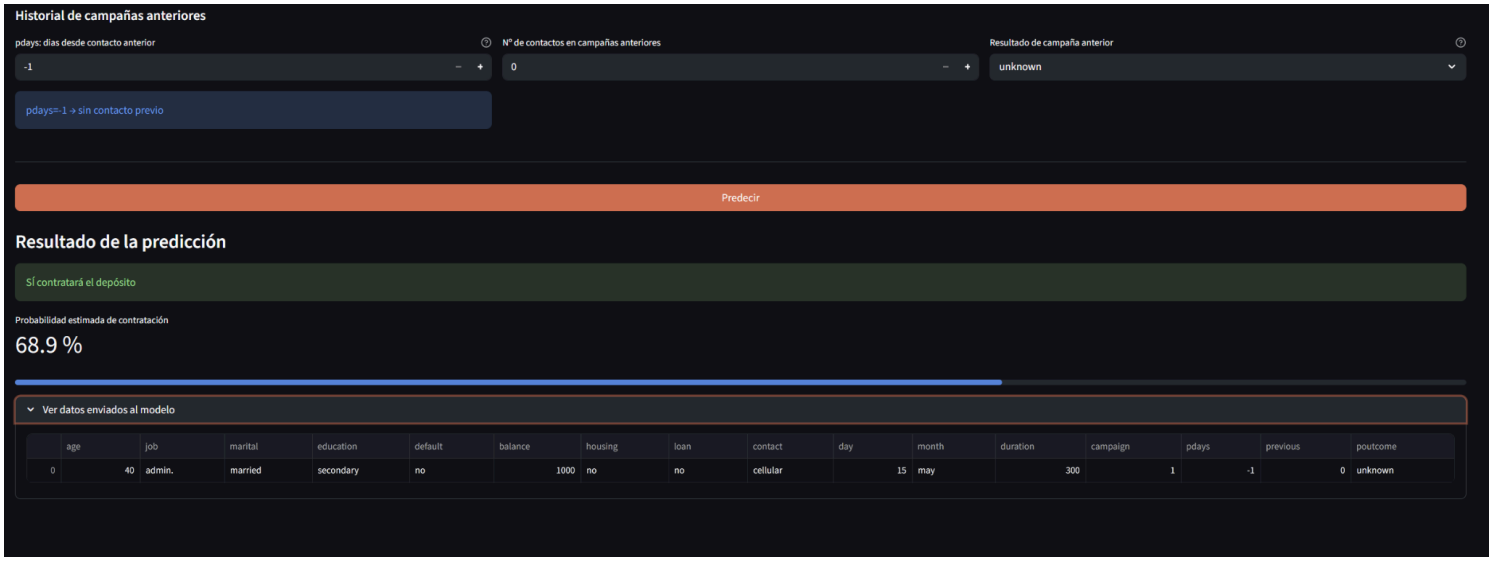
print('=' * 50)
print('VERIFICACIÓN PARA DEMO STREAMLIT')
print('=' * 50)
print(f'Instancia 1 (age=36, technician, duration=441s) → {"yes" if pred_1 else "no"}')
print(f'Instancia 2 (age=41, entrepreneur, duration=438s) → {"yes" if pred_2 else "no"}')
print()
print('Estas mismas instancias deben producir el mismo resultado en la app Streamlit.')
```

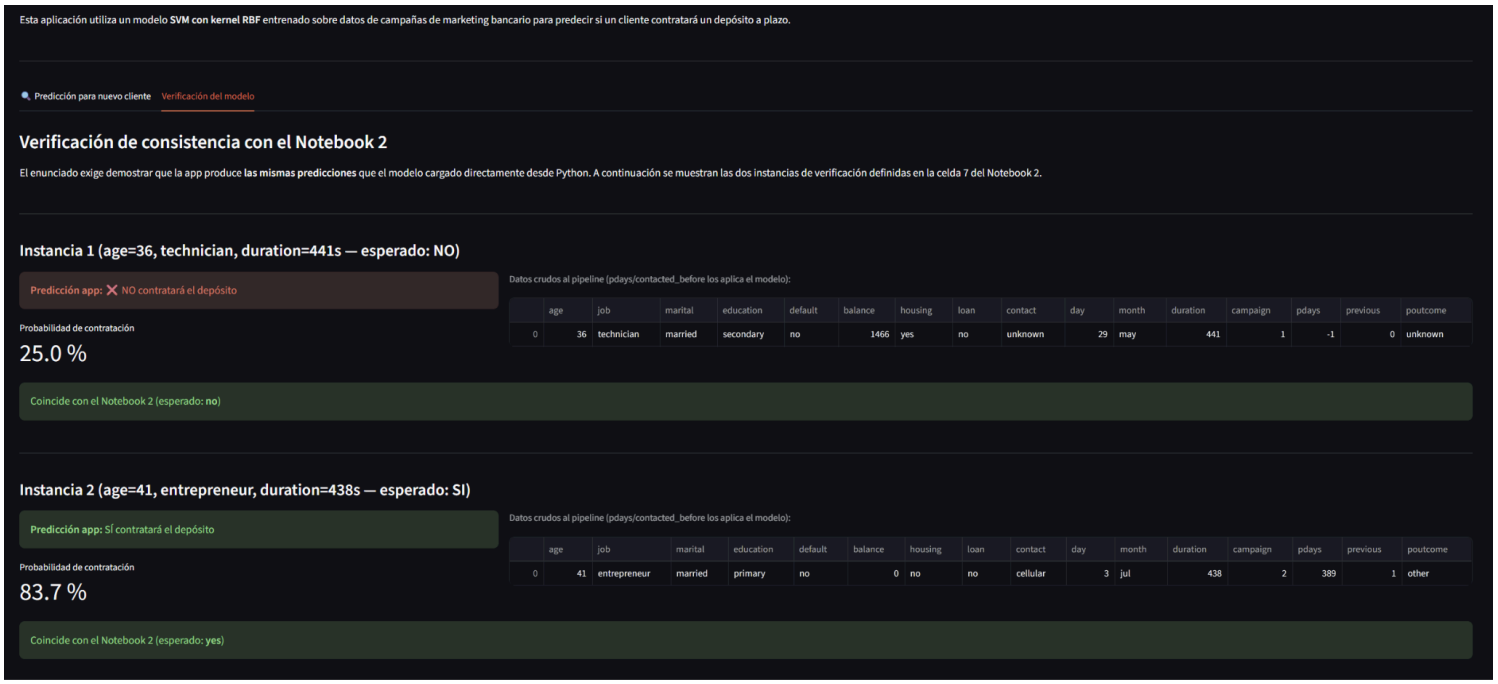
```
==================================================
VERIFICACIÓN PARA DEMO STREAMLIT
==================================================
Instancia 1 (age=36, technician, duration=441s) → no
Instancia 2 (age=41, entrepreneur, duration=438s) → yes

Estas mismas instancias deben producir el mismo resultado en la app Streamlit.
```

Streamlint:




Historial de campañas anteriores
pdays: días desde contacto anterior
-1
pdays=-1 → sin contacto previo
Nº de contactos en campañas anteriores
0
Resultado de campaña anterior
unknown
Predecir
Resultado de la predicción
Sí contratará el depósito
Probabilidad estimada de contratación
68.9 %
Ver datos enviados al modelo
age job marital education default balance housing loan contact day month duration campaign pdays previous poutcome
0 40 admin. married secondary no 1000 no no cellular 15 may 300 1 -1 0 unknown


Esta aplicación utiliza un modelo SVM con kernel RBF entrenado sobre datos de campañas de marketing bancario para predecir si un cliente contratará un depósito a plazo.
Predicción para nuevo cliente
Verificación del modelo
Verificación de consistencia con el Notebook 2
El enunciado exige demostrar que la app produce las mismas predicciones que el modelo cargado directamente desde Python. A continuación se muestran las dos instancias de verificación definidas en la celda 7 del Notebook 2.
Instancia 1 (age=36, technician, duration=441s — esperado: NO)
Predicción app: ❌ NO contratará el depósito
Probabilidad de contratación
25.0 %
Coincide con el Notebook 2 (esperado: no)
Datos crudos al pipeline (pdays/contacted_before los aplica el modelo):
age job marital education default balance housing loan contact day month duration campaign pdays previous poutcome
0 36 technician married secondary no 1466 yes no unknown 29 may 441 1 -1 0 unknown
Instancia 2 (age=41, entrepreneur, duration=438s — esperado: SI)
Predicción app: Sí contratará el depósito
Probabilidad de contratación
83.7 %
Coincide con el Notebook 2 (esperado: yes)
Datos crudos al pipeline (pdays/contacted_before los aplica el modelo):
age job marital education default balance housing loan contact day month duration campaign pdays previous poutcome
0 41 entrepreneur married primary no 0 no no cellular 3 jul 438 2 389 1 other